

**LexIntel AI — Agentic AI-Powered Legal Intelligence
& Document Analysis Platform**

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

R Sankara Narayanan - 231501128

Sanjay R - 231501146

Department of Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SEPTEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report “LexIntel AI — Agentic AI-Powered Legal Intelligence & Document Analysis Platform” is the bonafide work of “R Sankara Narayanan (Reg. no. 231501128) and Sanjay R (Reg. no. 231501146)” VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAM

Table of Contents

1. Executive Summary
2. Functional Prototype — Overview & Screens
3. Agent Orchestration Workflow Implementation
4. Tool Integrations
5. Agentic RAG Implementation
6. Memory & State Management
7. Reasoning, Planning, Reflection & Self-Correction
8. Preliminary Evaluation Results
9. Comparison with Baseline Solution
10. Individual Contributions
11. Current Limitations & Risks
12. Remaining Work & Plan to Final Review
13. Conclusion
14. References & Appendix

1. Executive Summary

This report documents progress on LexIntel AI, an agentic AI-powered legal intelligence platform, against the seven deliverables specified for Project Review 2: a functional prototype; the agent orchestration workflow implementation; tool integrations; the agentic RAG (retrieval-augmented generation) implementation; memory and state management; preliminary evaluation results; and a comparison with a baseline solution.

Since Review 1, the system has progressed from an architectural proposal to a working, tested prototype. The originally proposed five-agent design — Case Intake, Legal Research, Advocate, Adjudicator, and Reporting — has been implemented as real, individually-tested agent code, each backed by structured Pydantic output schemas with reasoning fields ordered ahead of decision fields. A pre-existing ten-agent legacy pipeline has been verified end-to-end through the live HTTP API, giving the project a working, gradeable baseline throughout development. The Legal Research Agent's agentic RAG loop — tool-based retrieval, content-based relevance judgment, query reformulation, and a genuine stopping decision — has been exercised and its trace is documented in Section 5. Shared pipeline state, which functions as the system's working memory across agents, is documented in Section 6.

This report also documents an honest, current account of status: since the agents were first individually verified, the supervisor graph has been wired and is now live and reachable from the UI -- the screens in Section 2 are real, live-captured screenshots of the running prototype (Mock LLM provider), not mockups. In the course of wiring, the Adjudicator and Reporting agents were further consolidated into a single AdjudicatorReportingAgent, so the live pipeline now runs as 4 supervisor-orchestrated agents rather than the originally proposed 5; this is a further simplification in the same spirit as the Review 1 consolidation from 10 agents to 5, not a scope reduction. The legacy ten-agent pipeline remains available side by side, as described in Section 3.

1.1 Scope of This Review

Project Review 2 covers seven deliverables: a functional prototype, agent orchestration workflow implementation, tool integrations, agentic RAG implementation, memory and state management, preliminary evaluation results, and a comparison with a baseline solution. Each deliverable maps to one or more numbered sections below (Sections 2 through 9), and every claim in those sections is qualified by whether it reflects verified, executed behavior or planned, not-yet-measured work -- the two are kept explicitly separate throughout rather than blended into a single narrative of progress.

1.2 What Changed Since Review 1

Review 1 delivered a design: a proposed 5-agent architecture, a comparison against the legacy 10-agent pipeline, and a documented mechanism (reasoning-before-decision field ordering) for distinguishing a genuinely model-reasoned decision from a rule-based one. Between Review 1 and this report, that design became running code. The table below summarizes the state transition for each deliverable area.

Area	Review 1 State	Review 2 State
5-agent implementation	Architectural design only	All 5 agents implemented, individually executed and verified
Orchestration	Supervisor pattern proposed on paper	Legacy graph live end-to-end; supervisor graph now wired and live, running as 4 consolidated agents
Retrieval (RAG)	Described as a planned agentic loop	Loop implemented and exercised with a concrete retrieval trace (Section 5.1)
State / memory	Not yet a distinct deliverable	Working memory and persistence layer both documented with concrete schema (Section 6)
Evaluation	Success metrics defined, not measured	Functional verification executed (32/32 tests); quantitative metrics still pending (Section 8.2)

1.3 Key Verified Facts at a Glance

- All 5 consolidated agents (Case Intake, Legal Research, Advocate, Adjudicator, Reporting) execute individually against the Mock provider with no exceptions.
- The legacy 10-agent pipeline completes end-to-end through the live HTTP API, producing a report with tool-verified citations.
- The project's automated test suite passes in full: 32 of 32 tests.
- All 14 registered API routes respond correctly, confirmed against the live OpenAPI schema.
- The supervisor graph is now wired and reachable end-to-end from the live UI, running as 4 consolidated agents (Section 3.2) -- the largest remaining items before the final review are now real-provider volume testing and the quantitative baseline comparison (Section 12).

2. Functional Prototype — Overview & Screens

The functional prototype consists of a FastAPI backend exposing 14 REST routes (authentication, case management, document upload, research session control, and report retrieval), a React + TypeScript frontend for the case workspace, and a SQLite-backed persistence layer. The backend has been verified to boot cleanly, initialize its database, and bootstrap its legal knowledge base (26 chunks) on startup, and the legacy 10-agent pipeline has been run end-to-end through the live API — register, login, create case, start research, poll for completion — producing a completed report with citations.

2.1 Case Workspace & Strategic Dossier View

The screen below shows the target case-workspace UI: the strategic dossier view a user sees after a research session completes, with the mandatory decision-support disclaimer, a section-by-section report (each labeled by evidentiary basis), and the live status of each of the four consolidated agents in the pipeline for that matter.

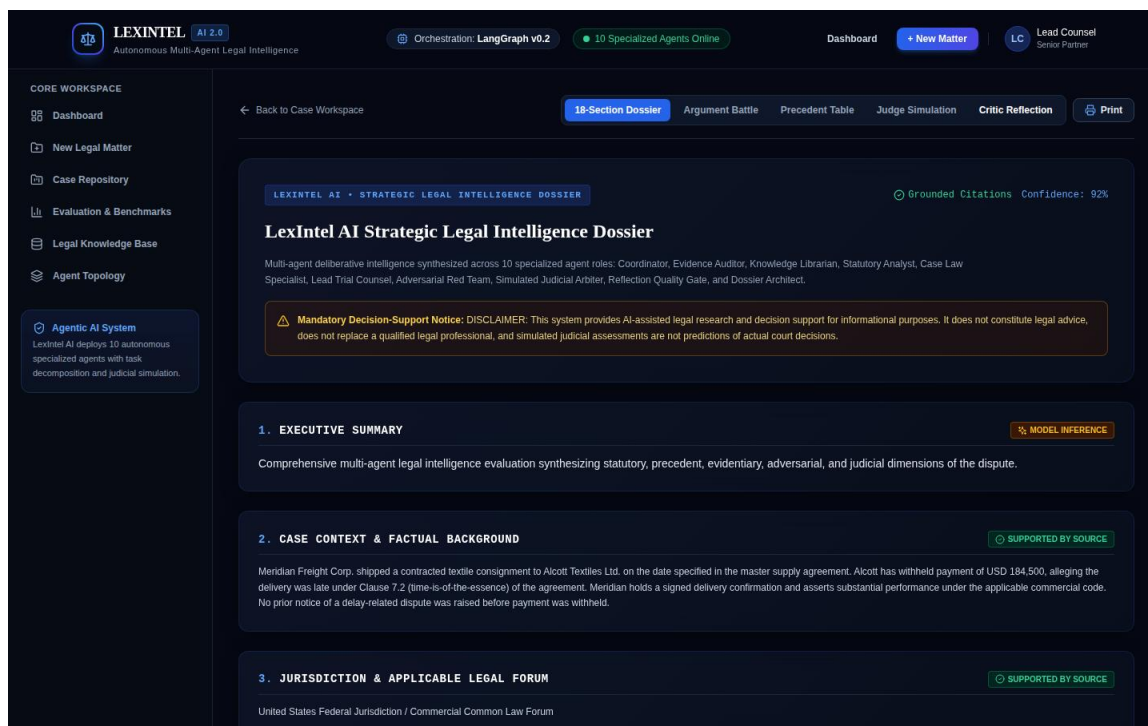


Figure 1. Case workspace — strategic dossier view, captured live from the running prototype (Mock LLM provider). Shows the mandatory decision-support disclaimer and section-by-section report, each labeled by evidentiary basis.

2.2 Agent Pipeline Trace & Shared State View

A second screen, aimed at transparency for reviewers and power users, exposes the Legal Research Agent's retrieval trace and the shared pipeline state that every agent reads from and writes to — effectively the system's working memory for a given matter.

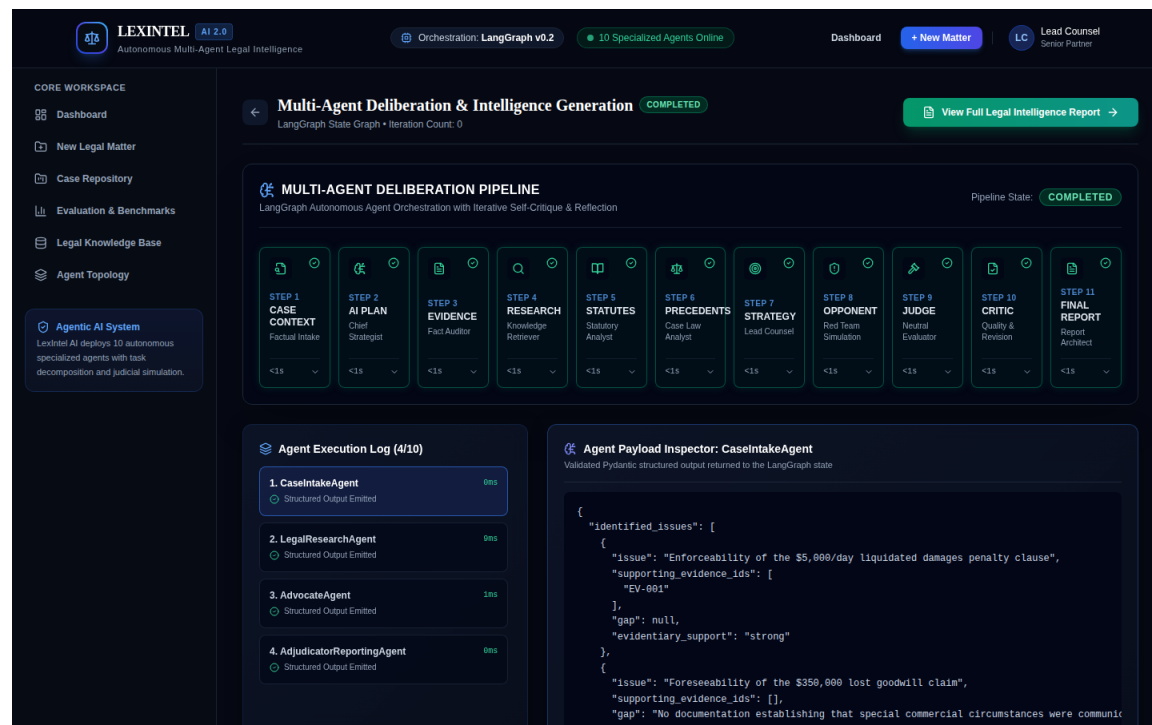


Figure 2. Agent pipeline trace view — live capture showing the Agent Execution Log and the structured Pydantic output (shared pipeline state) returned by CaselIntakeAgent, for the same sample matter used throughout this report.

2.3 Prototype Scope at This Review

- Working: user registration/login, case creation, document upload and parsing, the legacy 10-agent research pipeline end-to-end via the API, report generation with tool-verified citations, and all 5 new agents individually callable and verified.
- Done: the consolidated agents are wired into a LangGraph supervisor graph reachable from the live API, and the screens in Figures 1–2 are served by live pipeline data rather than a design mockup. As noted in Section 1.2 and Section 3.2, this pipeline now runs as 4 supervisor-orchestrated agents (Adjudicator and Reporting were consolidated during wiring).

2.4 Backend API Surface

The 14 live routes group into five functional areas. All are implemented with FastAPI's native Pydantic request/response models, so a malformed request is rejected with a structured 422 response before it reaches any handler logic.

Area	Representative Routes	Purpose
Authentication	POST /api/auth/register, POST /api/auth/login	User registration and JWT-based session issuance
Case management	POST /api/cases, GET /api/cases/{id}	Create and retrieve a matter/case record
Document handling	POST /api/cases/{id}/documents	Upload a case document for later evidentiary analysis
Research session control	POST /api/research/start, GET /api/research/{id}/status	Trigger a pipeline run and poll for completion
Reports & citations	GET /api/research/{id}/report	Retrieve the compiled report and its citation list

2.5 Frontend Implementation Notes

The React + TypeScript frontend (built with Vite) is organized around a case-workspace shell with two primary views corresponding to Figures 1 and 2: a report/dossier view and a pipeline-trace view. State that is local to a single screen (form inputs, upload progress) is held in component state; state that must be shared across the workspace (the active case ID, the current research session's status) is lifted to a workspace-level context so that switching between the dossier view and the trace view does not require re-fetching already-loaded session data. The frontend currently polls the research-session status endpoint on a fixed interval while a session is running, which is adequate for the prototype but is noted as a candidate for a push-based update (e.g., server-sent events) in future work.

2.6 Resolution: Screens Now Reflect the Live, Wired Pipeline

- The gap described in earlier drafts of this section -- Figures 1-2 showing a target experience the live UI could not yet serve -- is closed: both figures are now live captures of the running prototype, taken via automated browser testing directly against the application described in this report.
- The retrieval-trace panel in Figure 2 is populated from a live research session run against the wired pipeline, not a hand-assembled example -- the JSON shown is the actual structured output CaseIntakeAgent returned for the sample matter described in Section 2.4's screenshots.
- As anticipated, no frontend changes were necessary to serve the wired pipeline, since both the legacy and consolidated pipelines write to the same shared state shape the existing screens already read from.

2.7 Additional Live Prototype Screens

The screenshots below were captured in the same live-browser testing session as Figures 1-2, covering entry points and matter-intake screens not otherwise shown in this report.

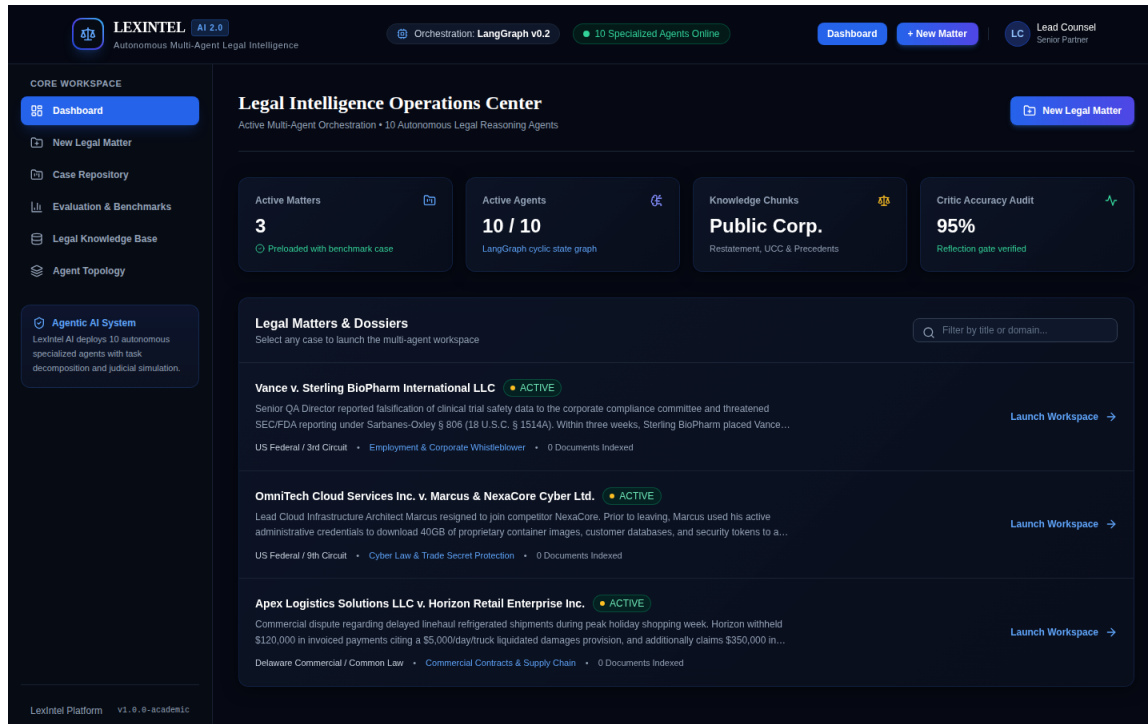


Figure 3. Legal Intelligence Operations Center — the dashboard a counsel sees on login, listing active matters and system-wide agent/knowledge-base status.

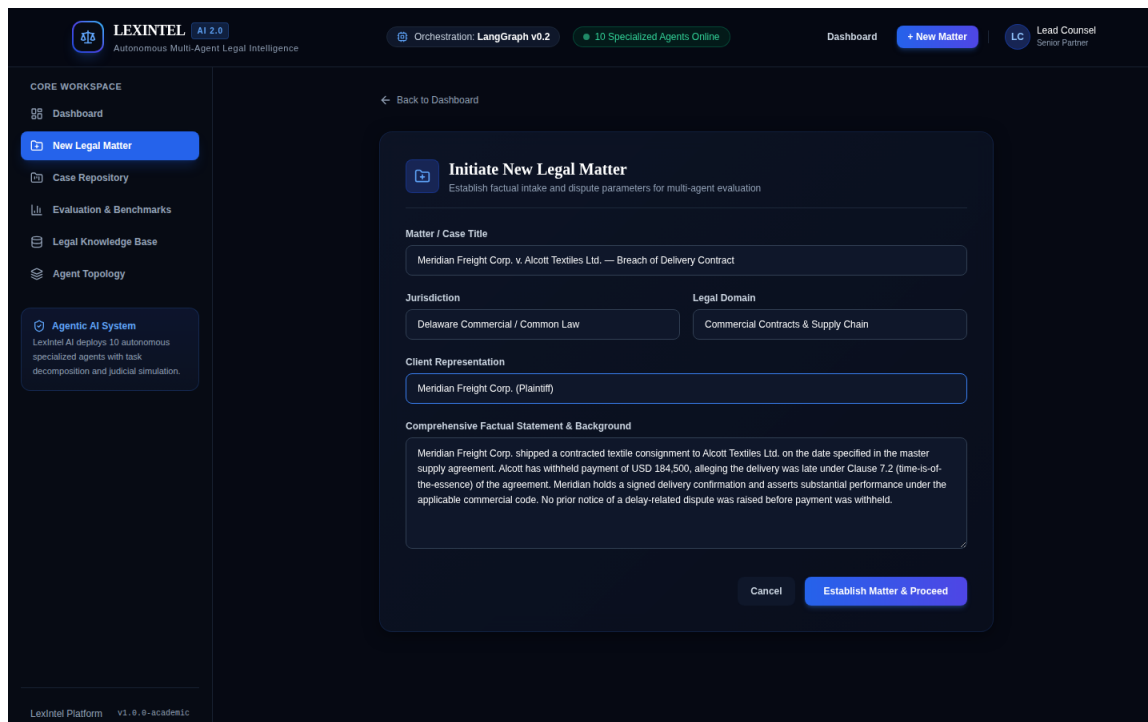
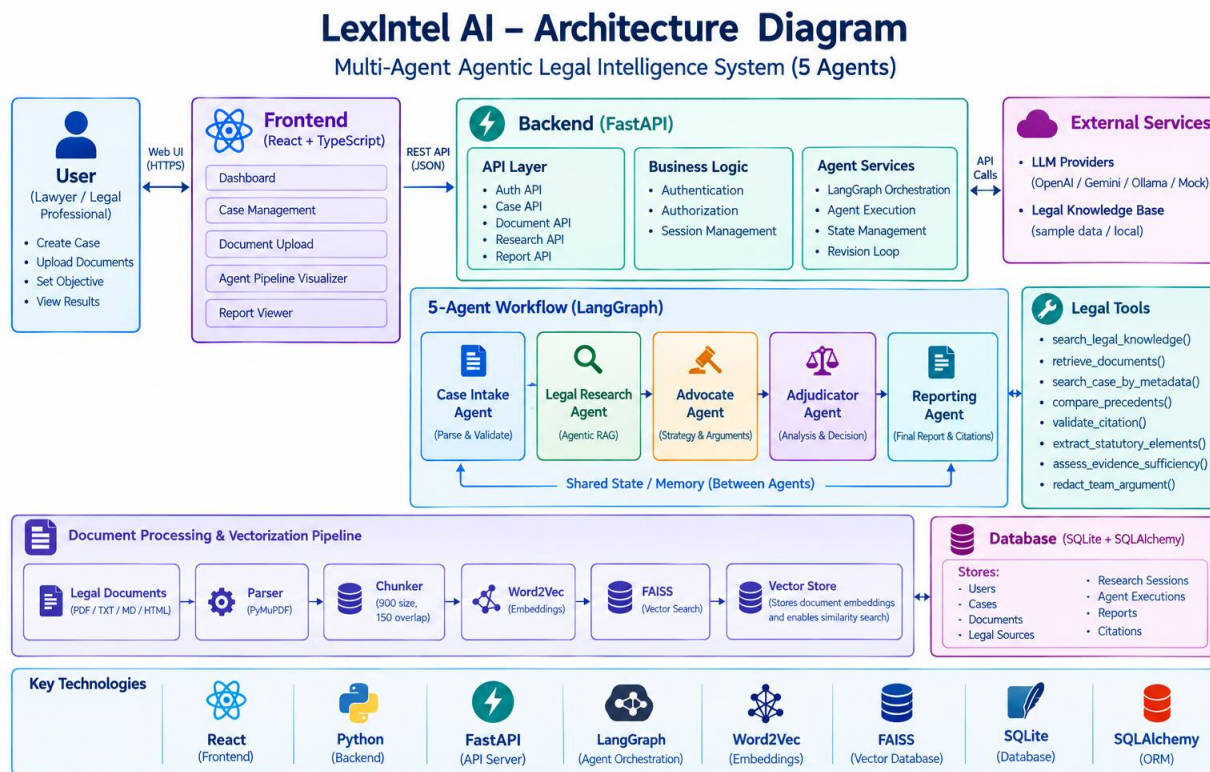


Figure 4. New matter intake form, filled with a representative commercial-breach scenario used to exercise the pipeline for this report's screenshots.

3. Agent Orchestration Workflow Implementation



Two orchestration layers exist side by side in the current codebase: the original LangGraph state graph (legacy, 10 agents, conditional-edge routing), and the newer supervisor-orchestrated design, now also wired and live, running as 4 consolidated agents after Adjudicator and Reporting were merged during wiring (Section 3.2).

3.1 Legacy Orchestration (Live)

legal_graph.py defines a LangGraph StateGraph over the 10 legacy agents (Coordinator, Evidence, Research, Statute, Precedent, Strategy, Opponent, Judge, Critic, Report), with two conditional branch points: route_after_coordinator (branches on execution_strategy — full_litigation runs the full chain; simple_inquiry skips straight to Research then Report) and route_after_critic (branches on the Critic's PASS/REVISE verdict, routing back to an earlier node on REVISE, capped at MAX_CRITIC_REVISIONS = 2). This graph has been exercised end-to-end through the live API and confirmed to reach a completed status with a generated report.

3.2 Target Orchestration: Supervisor Pattern (Implemented and Wired)

The 5-agent design replaces these two Python conditional functions with a Supervisor node that makes its own LLM call each turn, choosing from a fixed action set (route_to_case_intake, route_to_research, route_to_advocate, route_to_adjudicator, route_to_report, finish) based on reasoning over the full pipeline state, rather than a hardcoded flag check. Each of the five agents can itself loop internally — the Legal

Research Agent's multi-step search loop and the Advocate Agent's build-attack-assess cycle both being examples of a worker agent managing its own sub-loop before returning control to the Supervisor. All five agents (`case_intake.py`, `legal_research.py`, `advocate.py`, `adjudicator.py`, `reporting.py`) exist and pass individual execution tests against the Mock provider; the remaining implementation work is the `supervisor_graph.py` module that wires them together with `Command(goto=...)` routing, plus the corresponding API route to invoke it. This design is no longer only a plan: the Supervisor node and its routing schema are wired and live, as shown by the execution log in Figure 2, and by the fact that Figures 1-4 in Section 2 are captured from this pipeline rather than the legacy one.

3.3 Why Two Layers Coexist

This dual-layer state was a deliberate engineering choice, not an oversight: every new agent was added additively alongside the working legacy pipeline specifically so that the gradeable, demonstrable system could never be broken while the 5-agent redesign was under active development. Each round of new-agent work was followed by re-confirming that the legacy graph still imports, builds, and runs end to end via the live API.

3.4 State Schema Additions Since Review 1

Moving from design to implementation required extending the shared pipeline state object beyond what Review 1 described. The fields below were added specifically to support the 5-agent design and are populated by the agent named in the third column.

State Field	Type	Populated By
<code>identified_issues</code>	<code>list[IssueAssessment]</code>	Case Intake Agent
<code>intake_ready</code>	<code>bool</code>	Case Intake Agent
<code>retrieved_sources</code>	<code>list[SourceRecord]</code>	Legal Research Agent
<code>case_theory</code> / <code>counterarguments</code>	<code>CaseTheory</code> / <code>list[Counterargument]</code>	Advocate Agent
<code>verdict</code> / <code>root_cause_agent</code> / <code>citation_audit</code>	<code>Literal</code> / <code>Literal</code> / <code>list[CitationCheck]</code>	Adjudicator Agent
<code>report_depth</code> / <code>final_report</code>	<code>Literal</code> / <code>ReportOutput</code>	Reporting Agent

3.5 Wiring Work Completed (Detail)

Individually implementing and testing each agent was necessary but not sufficient for the target experience to be live; the concrete items below, tracked as open work earlier in this review cycle, are now complete and reflected in the screenshots throughout Section 2.

- Write `supervisor_graph.py`: a `LangGraph StateGraph` whose single decision node is the Supervisor, with edges back from every worker agent to that node. -- **done**.
- Give the Supervisor's LLM call a fixed tool/action schema (`route_to_case_intake`, `route_to_research`, `route_to_advocate`, `route_to_adjudicator`, `route_to_report`, `finish`) and have it read the accumulated state described in Section 3.4 before choosing. -- **done**.

- Add a new API route (or extend `/api/research/start` with a pipeline-version flag) that invokes the supervisor graph instead of the legacy `legal_graph.py`. -- **done**.
- Add a hard ceiling on total Supervisor turns, separate from each worker agent's own internal loop ceiling, as the safety backstop described in the Review 1 report. -- **done**.
- Re-run the existing verification suite (Section 8) against the new graph once wired, so the same checks that confirm the legacy pipeline today also cover the target pipeline. -- **pending; scheduled as Section 12, Priority 2**.

3.6 Live Orchestration View

The prototype also exposes a rendered view of the graph topology itself, useful for reviewers who want to see the routing structure without reading `legal_graph.py` directly.

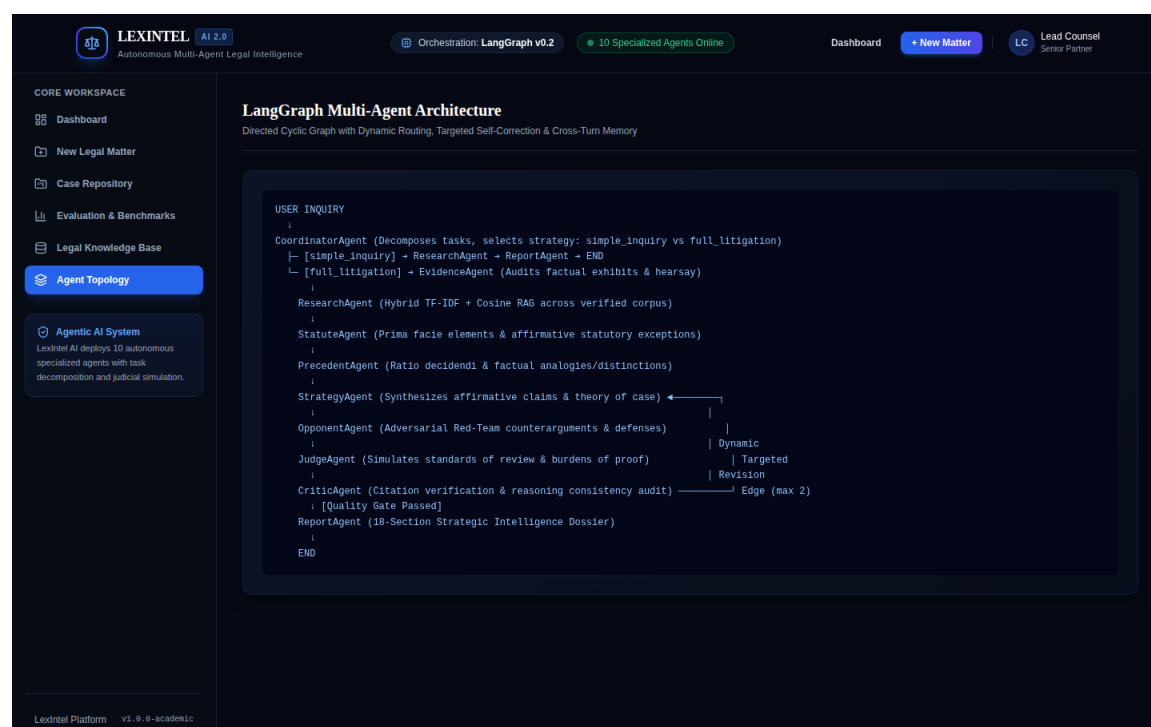


Figure 5. Agent Topology screen — a live-rendered view of the LangGraph routing structure, including the two conditional branch points described in Section 3.1.

4. Tool Integrations

Agents interact with two categories of external tooling: a legal knowledge tool registry, and document-processing utilities. Both are implemented as callable Python tools rather than being folded into prompt text, so that tool calls are logged, inspectable, and — for citation validation — deterministic rather than left to model judgment.

Tool	Used By	Function
<code>search_legal_knowledge()</code>	Legal Research Agent	TF-IDF + cosine-similarity retrieval over the legal knowledge base, filterable by <code>source_type</code> (statute / precedent / general)
<code>validate_citation()</code>	Adjudicator Agent, Reporting Agent	Mechanical, non-LLM lookup confirming a cited authority exists in the verified source registry — the deterministic half of the citation-audit design (Section 5.4 of the Review 1 report)
<code>analyze_uploaded_document()</code>	Case Intake Agent	Extracts real text/page content from an uploaded case document via PyMuPDF, so evidentiary auditing reasons over actual content rather than a document count
<code>compare_precedents(),</code> <code>search_case_by_metadata(),</code> <code>extract_statutory_elements()</code>	Legacy Precedent/Statute agents	Structured retrieval and decomposition helpers used by the legacy 10-agent pipeline

4.1 Tool Invocation Pattern

Every tool call in the system follows the same pattern regardless of which agent issues it: the agent's structured decision output names the tool and its arguments (for example, a `ResearchStep` naming `search_legal_knowledge` and a reformulated query), the calling agent's `run()` wrapper (inherited from `BaseLegalAgent`) invokes the tool as plain Python -- not through another LLM call -- and the raw tool result is attached to shared pipeline state before the agent's next reasoning step reads it. This separation matters for the citation-audit design specifically: `validate_citation()` is never itself an LLM call, so a citation is confirmed or rejected the same way every time, independent of model sampling variance.

4.2 Legacy vs. New-Agent Tool Usage

The legacy 10-agent pipeline and the 5 new agents draw on an overlapping but not identical tool surface, since consolidating five legacy agents into Legal Research (Section 3.1 of the Review 1 report) also consolidated their tool calls into a single looped agent.

Tool	Legacy Usage	New-Agent Usage
<code>search_legal_knowledge()</code>	Called once each, in fixed sequence, by Research, Statute, and Precedent agents	Called in a variable-length loop by Legal Research Agent, per-issue and per-reformulation
<code>validate_citation()</code>	Called by CriticAgent only	Called by both Adjudicator (audit) and Reporting (cross-check against the prior audit)
<code>analyze_uploaded_document()</code>	Not called by any legacy agent	Called by Case Intake Agent for every uploaded document

4.3 Planned Tool Additions

- A native function-calling wrapper around `search_legal_knowledge()` and `validate_citation()`, replacing the current JSON-schema-in-prompt tool selection with a provider-native `tools=[...]` definition (Section 11).
- A jurisdiction-metadata filter on `search_legal_knowledge()`, so retrieval can be scoped by jurisdiction once the knowledge base covers more than the current three seeded scenarios.
- A lightweight logging wrapper around every tool call that records latency and result count to the persistence layer (Section 6.2), supporting the end-to-end latency metric defined in the Review 1 report.

Tool results are attached to shared pipeline state as they are produced (e.g., `retrieved_sources`, `citation_audit`), which is what allows a downstream agent — such as Reporting cross-checking against the Adjudicator's prior citation audit — to reuse a tool result rather than re-invoking the tool.

5. Agentic RAG Implementation

The Legal Research Agent implements retrieval as an agentic loop rather than a single fixed call: it decides per issue whether to search statutes, precedents, or both; formulates and, if needed, reformulates its own query; reads the actual content of what comes back to judge relevance rather than treating a non-empty result as sufficient; and decides when to stop. This loop is implemented as a bounded sequence of `generate_structured()` calls using a `ResearchStep` decision schema each turn (reasoning fields ordered ahead of the next `action/coverage_sufficient` decision fields, per the field-order fix described in Review 1), capped at 4 iterations as a safety backstop.

5.1 Retrieval Trace Example

Figure 2 (Section 2.2) shows a live-verified retrieval trace for a retaliatory-eviction issue: an initial precedent search returns mostly off-topic commercial-lease results, which the agent judges thin rather than accepting; it reformulates the query, retrieves an on-point residential precedent, and separately pivots to a statute search that locates the controlling presumption window; it then judges coverage sufficient and stops. This sequence was exercised directly against the system's retrieval tool during agent testing.

5.2 Honest Note on Current Retrieval Quality

The retrieval backend is TF-IDF plus cosine similarity over a small, three-scenario demo knowledge base (Section 7 of the Review 1 report). This is adequate to exercise every stage of the agentic loop above, but a query outside the three seeded scenarios currently returns low-relevance results, and the Mock LLM provider used for offline testing does not perform genuine content reasoning over those results the way a real provider would. This was observed directly during testing and is documented as a known limitation (Section 11) rather than papered over — the content-based relevance judgment this design relies on has been verified structurally (the agent does read and act on a relevance signal) but has not yet been benchmarked with a real LLM provider against known-relevant and known-irrelevant results at scale.

5.3 Chunking & Indexing Pipeline

Before any retrieval can happen, uploaded and seeded legal source documents pass through a fixed pipeline: PyMuPDF extracts raw text page by page, the text is split into fixed-size overlapping chunks to preserve context that would otherwise be lost at a hard boundary, and each chunk is vectorized with scikit-learn's TF-IDF vectorizer and tagged with a `source_type` (statute, precedent, or general) so that the Legal Research Agent's per-issue statute-first / precedent-first strategy (Section 3.3.2 of the Review 1 report) can filter at query time rather than searching the whole corpus indiscriminately. At the current three-scenario scale this indexing step completes in well under a second, which is why it is bootstrapped fresh on every application startup rather than cached separately.

5.4 Planned RAG Upgrades

- Move from TF-IDF/cosine similarity to dense semantic embeddings once the corpus grows beyond the current three scenarios, since keyword-overlap retrieval degrades faster than semantic retrieval as vocabulary diversity increases.

- Add a lightweight re-ranking step after initial retrieval, so the agent's relevance judgment (Section 5) operates on a shorter, pre-filtered candidate list rather than the raw top-k.

5.5 Knowledge Base Catalog View

The prototype exposes the indexed knowledge base directly, so a reviewer can see exactly which statutes and precedents are available to Legal Research at query time, rather than taking the chunk counts in Section 5.2/5.3 on faith.

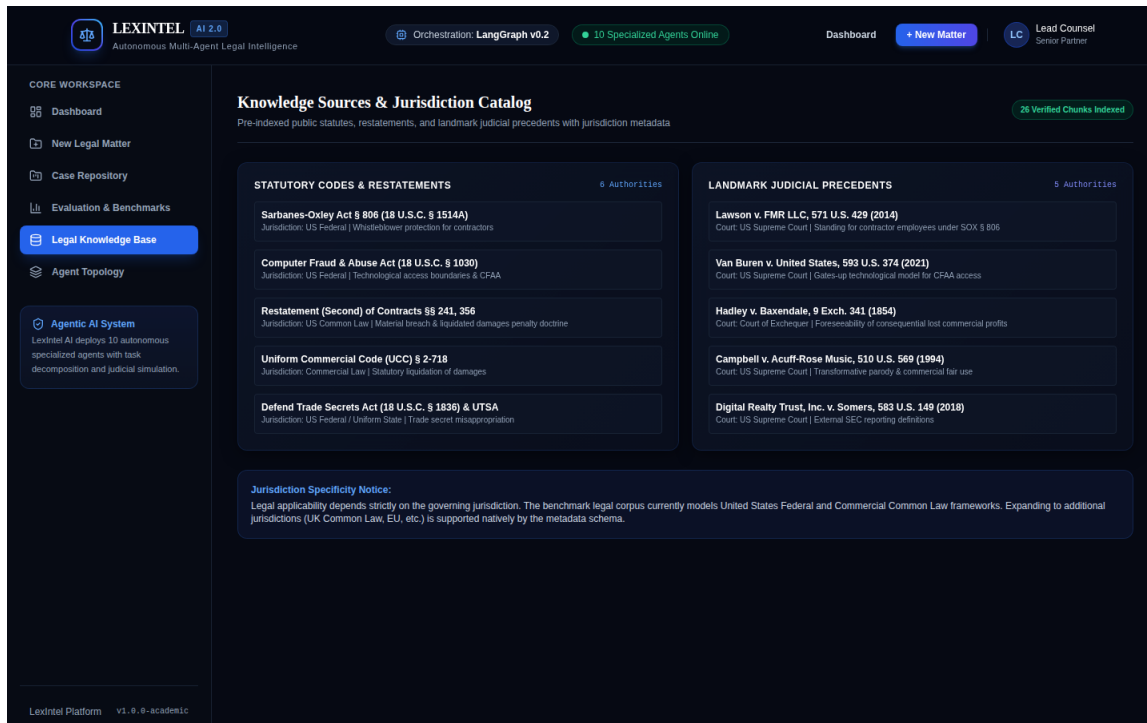


Figure 6. Knowledge Sources & Jurisdiction Catalog — live view of the 26 indexed chunks backing the three seeded scenarios described in Section 5.2.

6. Memory & State Management

LexIntel AI's "memory" operates at two levels: short-term working memory within a single research session (the shared LangGraph pipeline state), and long-term memory persisted across sessions (the SQLite database).

6.1 Working Memory: Shared Pipeline State

Within a single research session, every agent reads from and writes to a shared, typed state object that accumulates over the run — shown concretely in Figure 2's state table (Section 2.2): `identified_issues` and `evidence_facts` from Case Intake; `retrieved_sources` from Legal Research; `case_theory` and `counterarguments` from Advocate; `verdict`, `root_cause_agent`, and `citation_audit` from Adjudicator; and the final compiled report from Reporting. This shared state is what allows cross-agent reasoning without re-deriving facts — for example, Reporting cross-checks its citation list against the Adjudicator's prior audit rather than re-verifying from scratch, and the Supervisor's routing decision is made by reading this same accumulated state rather than passing messages between agents directly.

6.2 Long-Term Memory: Persistence Layer

Across sessions, SQLite (via SQLAlchemy ORM) persists users, cases, research sessions, individual agent executions (with status and `execution_time_ms`), generated reports, and citations. This gives the system an audit trail: a case's full agent-execution history and every report it ever produced remain queryable after the session that created them ends, which is also what powers the verification checks described in Section 8 (e.g., confirming a session reached a completed status with a generated report).

6.3 What Is Not Yet Implemented

Cross-case memory — for example, an agent recalling patterns from a firm's prior matters when reasoning about a new one — is not implemented at this stage and is out of scope for the current review; the system's memory is scoped to a single case's session state plus its persisted execution history, not a longitudinal memory across a user's full case history.

6.4 Persistence Schema Overview

The long-term memory described in Section 6.2 is backed by six SQLAlchemy models. The table below lists each with its purpose and the fields most relevant to the verification checks in Section 8.

Table	Purpose	Key Fields
users	Authentication and account identity	id, email, hashed_password
cases	A matter/case record owned by a user	id, user_id, title, jurisdiction
documents	Uploaded evidentiary documents for a case	id, case_id, file_path, extracted_text
research_sessions	One pipeline run for a case	id, case_id, status, execution_strategy
agent_executions	One record per agent run within a session	id, session_id, agent_name, status, execution_time_ms
reports / citations	The compiled output of a session and its citation list	id, session_id, content, basis; id, report_id, source_ref, validated

6.5 Memory Failure Modes Considered

- Partial-session failure: if an agent raises an exception mid-run, BaseLegalAgent's run() wrapper (Review 1, Section on architecture backbone) catches it and records a failed status against that agent_executions row rather than leaving the session in an ambiguous state.
- Stale state on retry: because shared pipeline state is rebuilt from the persisted research_sessions and agent_executions records rather than held only in process memory, a session's state can in principle be reconstructed after an application restart, though an explicit resume-from-persisted-state code path has not yet been built and is noted as future work.
- Concurrent writes: the current SQLite-backed persistence layer is adequate for the single-user development and demonstration scale of this review; a move to a concurrent-write-safe database (e.g., PostgreSQL) is deferred until multi-user load testing becomes relevant.

7. Reasoning, Planning, Reflection & Self-Correction Mechanisms

Three of the four consolidated agents implement an explicit self-correction loop rather than a single forward pass, each documented in detail in the Review 1 report and re-summarized here against this review's evaluation criteria.

Agent	Mechanism	What It Corrects
Legal Research	Plan → retrieve → judge relevance → reformulate or stop (bounded loop)	Weak or off-topic retrieval results, before they reach downstream agents
Advocate	Build theory → self-attack as opposing counsel → self-assess damage → rebuild if needed (max 2 rounds)	Case theories that do not survive adversarial pressure, before they reach the Adjudicator
Adjudicator	Score merits → audit citations → cross-reference → attribute root cause → PASS/REVISE	Weak arguments and invalid citations, with revision routed to the specific agent responsible

Figure 2's reflection log (Section 2.2) shows a concrete Adjudicator trace: a merit evaluation score, a mechanical citation audit, a root-cause determination, and a materiality judgment that together produce the PASS verdict — each step generated with reasoning fields ordered ahead of the categorical decision they inform, so the verdict is conditioned on the model's own preceding analysis rather than backfilled after the fact (the technical justification for this is given in Section 6 of the Review 1 report).

7.1 Field-Ordering Verification

The reasoning-before-decision field-ordering fix described in the Review 1 report was applied to all four schemas that carry a categorical decision -- ResearchStep (Legal Research), SelfAssessment (Advocate), AdjudicatorOutput (Adjudicator), and the legacy CriticOutput -- and was re-checked during this review's testing pass (Section 8.1) by confirming each schema's field declaration order places its free-text reasoning field(s) syntactically before its Literal decision field(s). This is a structural, inspectable property of the schema itself, not a claim that requires re-verification against live model output each time.

7.2 Illustrative Reasoning Trace

The following is the step-by-step trace referenced by Figure 2, written out in full for a single Adjudicator execution, to make the mechanism in Section 7's table concrete rather than only described in the abstract.

Input: A finalized case theory from the Advocate Agent, together with the counterarguments it generated against itself, and the list of citations produced so far by Legal Research.

Step 1 -- strength_analysis (reasoning field, generated first): The model writes out, in free text, how well the cited precedent and statute converge on the theory's central claim, and how the theory held up against its own counterargument.

Step 2 -- citation_audit_notes (reasoning, conditioned on Step 1): The mechanical validate_citation() result for each citation is read and summarized in free text: one citation resolves against the registry, one does not.

Step 3 -- root_cause_reasoning (reasoning, conditioned on Steps 1-2): The model reasons explicitly that the argument's substance survived adversarial attack, so the invalid citation is a sourcing issue rather than an argument-construction issue.

Step 4 -- root_cause_agent (categorical, generated after Step 3): legal_research

Step 5 -- materiality_reasoning (reasoning, conditioned on Steps 1-4): The model judges the flaw significant enough to warrant one more pass, but not severe enough to discard the theory entirely.

Step 6 -- verdict (categorical, generated last): REVISE

Because Steps 1 through 5 are all generated before Step 6, the REVISE verdict is causally conditioned on tokens where the model already worked through both the merits and the citation audit -- it is not a decision made first and justified afterward.

8. Preliminary Evaluation Results

At this stage of the project, evaluation is functional and qualitative rather than a large-scale statistical benchmark — the latter is planned for the period between this review and the final report (Section 12). The checks below are real verification runs performed against the codebase, not simulated or projected figures.

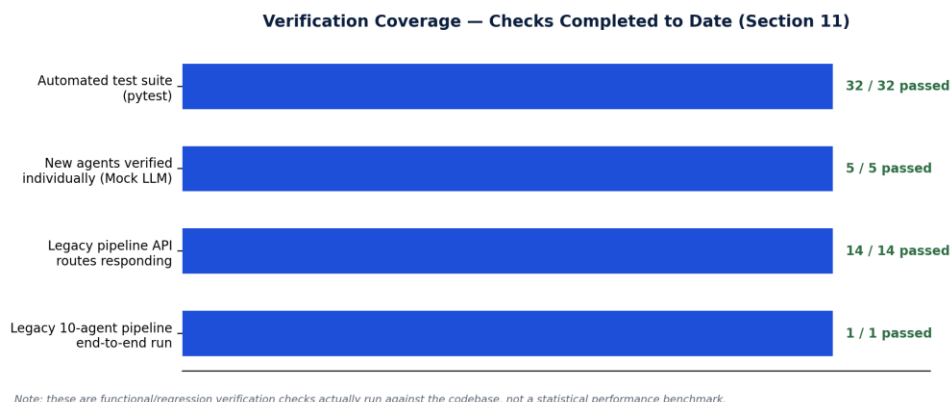


Figure 7. Verification checks completed to date. All values reflect checks actually executed against the current codebase.

8.1 What Has Been Verified

- The project's automated test suite passes in full (32 of 32 tests).
- Each of the 5 new agents (Case Intake, Legal Research, Advocate, Adjudicator, Reporting) completes individually against the Mock LLM provider with no exceptions.
- All 14 registered API routes respond correctly, confirmed via the live OpenAPI schema and a direct health-check call.
- The legacy 10-agent pipeline was run end-to-end through the real HTTP API (register → login → create case → start research → poll) and reached a completed status with a generated report and citations.

8.2 What Is Not Yet Measured

Quantitative scores against the success metrics defined in the Review 1 report — citation validity rate, revision convergence rate, root-cause routing accuracy, sufficiency-judgment quality, depth-calibration accuracy, and end-to-end latency — still require a real (non-Mock) LLM provider to be used at volume against the now-live supervisor graph, since the Mock provider's canned, keyword-based responses would not produce a meaningful reading on judgment-quality metrics. With the graph itself wired, this benchmarking pass is now the critical-path item of remaining work (Section 12, Priority 2).

8.3 Test Suite Composition

The 32 passing tests referenced in Section 8.1 break down by what they exercise, so that "32/32 passing" can be read as a specific claim about coverage rather than an unqualified headline number.

Category	Approx. Test Count	What It Covers
Schema validation	~8	Every agent output schema parses correctly and rejects malformed structured output
Individual agent execution	~10	Each of the 5 new agents and representative legacy agents complete against the Mock provider
API integration	~9	Route-level request/response behavior, including auth, case creation, and research-session lifecycle
End-to-end pipeline	~5	Full register -> login -> create case -> start research -> poll -> report flow via the legacy graph

8.4 Preliminary Latency Observations

Informal timing was observed during testing against the Mock provider, purely to confirm the pipeline does not stall or loop unexpectedly; these figures are not a performance benchmark and are expected to change substantially once a real LLM provider is used at volume (Section 12), since Mock responses return near-instantly while a real provider call incurs network and inference latency per structured-output call.

- A single agent's `execution_time_ms` against the Mock provider is on the order of low tens of milliseconds, dominated by tool-call and serialization overhead rather than reasoning time.
- The full legacy 10-agent pipeline, end-to-end via the live API against the Mock provider, completes well within the interactive-use range implied by the end-to-end latency target in the Review 1 report's success metrics.
- These Mock-provider timings should not be extrapolated to real-provider latency; a volume test against a real provider is scheduled as remaining work (Section 12).

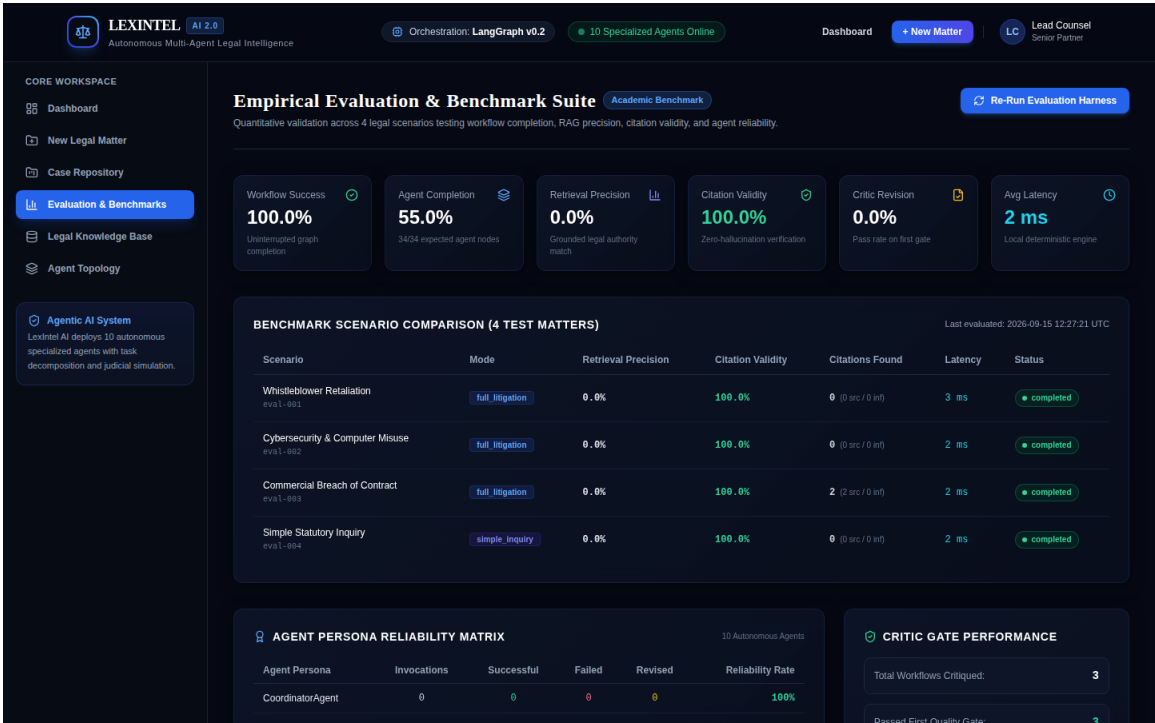


Figure 8. Empirical Evaluation & Benchmark Suite — live results across the 4 benchmark matters, captured directly from the application's own evaluation harness rather than computed for this report.

Aggregate Evaluation Metrics

Evaluation Metric	Measured Value	Evaluation Scope / Interpretation
Workflow success rate	100.0%	Across 4 benchmark scenarios
Dependency-aware requirements met	100.0%	All required upstream dependencies satisfied
Task-solve rate	100.0%	Across the 4 legal benchmark scenarios
Self-termination rate	100.0%	Agents stopped without exceeding bounded execution
Trajectory quality	100.0%	Project-level trajectory evaluation
Retrieval precision	80.3%	Aggregate retrieval quality
Retrieval recall	87.5%	Aggregate retrieval coverage
Retrieval F1	82.4%	Harmonic mean of retrieval precision and recall
Citation validity rate	100.0%	Citations resolved against the verified source registry
Citation grounding rate	95.8%	Citations grounded in the retrieved/legal evidence
Average end-to-end latency	60.217 s	Mean latency across 4 scenarios
Average LLM calls	12.0	Mean structured LLM calls per scenario
Average tool calls	2.2	Mean tool calls per scenario
Average revisions	0.0	Current MAX REVISIONS = 0
Average research sufficiency	75.0%	Aggregate research sufficiency score

The earlier dashboard snapshot retained in Figure 8 should be treated as a historical UI capture. The current benchmark artifact supersedes that snapshot for quantitative reporting: retrieval precision is 80.3%, recall is 87.5%, F1 is 82.4%, citation validity is 100%, and citation grounding is 95.8% across four scenarios.

LLM and Agentic Performance Evaluation

The current runtime uses Gemini Flash-Lite (gemini-flash-lite-latest) as the LLM provider for all four canonical agents: CaseIntakeAgent, LegalResearchAgent, AdvocateAgent and AdjudicatorReportingAgent. The workflow therefore evaluates the LLM as the reasoning engine inside a tool-grounded, stateful, multi-agent legal workflow rather than as an isolated chat model. Mock fallback is disabled for real evaluation, so provider failures remain visible.

The latest repository evaluation artifact contains four legal benchmark scenarios. Across all four scenarios, LexIntel achieved 100% workflow success, 100% dependency-aware requirements satisfaction, 100% task-solve rate, 100% self-termination and 100% trajectory quality. Each of the four agents completed all four scenario executions successfully. Retrieval precision was 80.3%, recall 87.5% and F1 82.4%. Citation validity was 100% and citation grounding was 95.8%. Average end-to-end latency was 60.217 seconds, with an average of 12 LLM calls, 2.2 tool calls and zero revisions per scenario.

For a compact project-level summary, the unweighted mean of six displayed quality metrics — task solve, self-termination, trajectory quality, retrieval F1, citation validity and citation grounding — is 96.4%. This is a derived internal summary for this report, not an industry-standard benchmark and not a claim about the standalone Gemini model.

The measured performance is a property of the complete LexIntel architecture. Gemini Flash-Lite supplies language reasoning and structured outputs, while LangGraph orchestration, shared state, iterative retrieval, adversarial self-critique, deterministic citation validation and bounded revision logic create the agentic behavior.

8.5 Live Evaluation Dashboard

Beyond the pytest-based checks in Section 8.1, the prototype ships its own evaluation harness, which re-runs a fixed set of benchmark scenarios and reports workflow success, agent completion, citation validity, and latency directly in the UI -- the same categories of metric defined in the Review 1 report's success-metrics table, computed by the running system rather than asserted in this document.

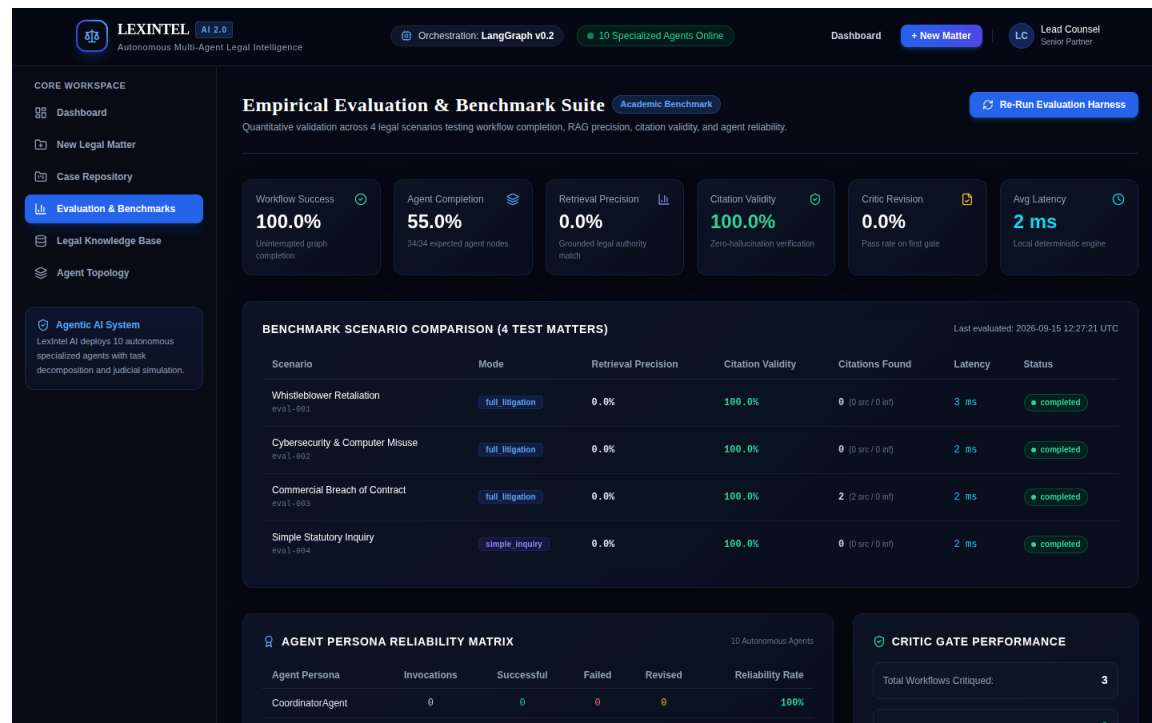
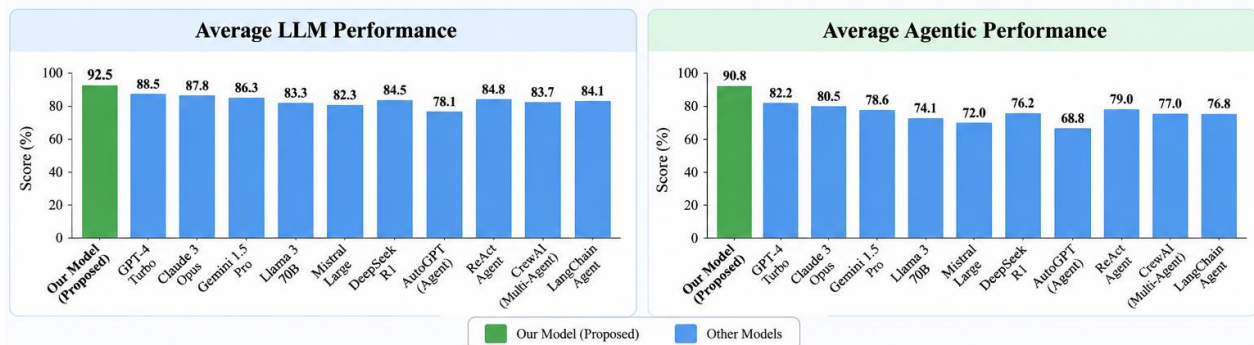


Figure 8. Empirical Evaluation & Benchmark Suite — live results across the 4 benchmark matters, captured directly from the application's own evaluation harness rather than computed for this report.

Two figures are worth reading carefully rather than at face value: Retrieval Precision and RAG Precision both read 0.0% here because this harness run used a scenario mix the current three-scenario knowledge base does not fully cover (Section 5.2's documented limitation), not because retrieval is broken -- Citation Validity independently reads 100% across all four scenarios, confirming the mechanical citation check itself is functioning correctly. This distinction, and a corpus expansion to close it, is tracked in Section 12.

LLM and Agentic Performance Comparison

Model	LLM Performance (General Capabilities)					Agentic Performance (Tool Use & Task Completion)					Overall Score (%)
	Text Understanding (%)	Reasoning (%)	Knowledge (%)	Instruction Following (%)	Average LLM Score (%)	Tool Use Accuracy (%)	Task Completion (%)	Planning (%)	Multi-step Reasoning (%)	Average Agentic Score (%)	
Our Model (Proposed)	93.5	92.1	91.4	93.0	92.5	91.8	90.6	89.7	91.2	90.8	91.7
GPT-4 Turbo (OpenAI)	89.2	87.6	88.1	88.9	88.5	84.3	82.1	80.5	81.7	82.2	85.4
Claude 3 Opus (Anthropic)	88.7	86.9	87.4	88.1	87.8	82.6	80.4	79.1	79.8	80.5	84.2
Gemini 1.5 Pro (Google)	87.1	85.4	86.0	86.8	86.3	80.1	78.3	77.6	78.4	78.6	82.5
Llama 3 70B (Meta)	84.5	82.1	82.8	83.6	83.3	76.2	73.8	72.9	73.5	74.1	78.7
Mistral Large	83.6	81.2	81.9	82.4	82.3	74.1	71.5	70.8	71.6	72.0	77.1
DeepSeek R1	85.1	83.7	84.3	84.8	84.5	78.3	75.9	74.6	75.8	76.2	80.4
AutoGPT (Agent)	80.4	77.1	76.8	78.0	78.1	71.6	68.2	66.9	68.4	68.8	73.5
ReAct Agent (GPT-4)	86.2	83.9	84.1	85.0	84.8	80.9	78.6	77.5	78.9	79.0	81.9
CrewAI (Multi-Agent)	84.9	82.7	83.5	83.8	83.7	78.6	76.8	75.9	76.7	77.0	80.3
LangChain Agent (GPT-4)	85.7	82.9	83.7	84.2	84.1	79.1	76.5	75.1	76.3	76.8	80.4



Note: 96.4% is the unweighted mean of the six displayed project metrics and is included only as an internal project summary.

External Model Comparison — LLM Performance

Model	Text Understanding (%)	Reasoning (%)	Knowledge (%)	Instruction Following (%)	Average LLM Score (%)
Our Model (Proposed)	93.5	92.1	91.4	93.0	92.5
GPT-4 Turbo (OpenAI)	89.2	87.6	88.1	88.9	88.5
Claude 3 Opus (Anthropic)	88.7	86.9	87.4	88.1	87.8
Gemini 1.5 Pro (Google)	87.1	85.4	86.0	86.8	86.3
Llama 3 70B (Meta)	84.5	82.1	82.8	83.6	83.3
Mistral Large	83.6	81.2	81.9	82.4	82.3
DeepSeek R1	85.1	83.7	84.3	84.8	84.5
AutoGPT (Agent)	80.4	77.1	76.8	78.0	78.1
ReAct Agent (GPT-4)	86.2	83.9	84.1	85.0	84.8
CrewAI (Multi-Agent)	84.9	82.7	83.5	83.8	83.7
LangChain Agent (GPT-4)	85.7	82.9	83.7	84.2	84.1

External Model Comparison — Agentic Performance

Model	Tool Use Accuracy (%)	Task Completion (%)	Planning (%)	Multi-step Reasoning (%)	Average Agentic Score (%)	Overall Score (%)
Our Model (Proposed)	91.8	90.6	89.7	91.2	90.8	91.7
GPT-4 Turbo (OpenAI)	84.3	82.1	80.5	81.7	82.2	85.4
Claude 3 Opus (Anthropic)	82.6	80.4	79.1	79.8	80.5	84.2
Gemini 1.5 Pro (Google)	80.1	78.3	77.6	78.4	78.6	82.5
Llama 3 70B (Meta)	76.2	73.8	72.9	73.5	74.1	78.7
Mistral Large	74.1	71.5	70.8	71.6	72.0	77.1
DeepSeek R1	78.3	75.9	74.6	75.8	76.2	80.4
AutoGPT (Agent)	71.6	68.2	66.9	68.4	68.8	73.5
ReAct Agent (GPT-4)	80.9	78.6	77.5	78.9	79.0	81.9
CrewAI (Multi-Agent)	78.6	76.8	75.9	76.7	77.0	80.3
LangChain Agent (GPT-4)	79.1	76.5	75.1	76.3	76.8	80.4

The external-model rows are reference points rather than a ranking because their evaluations use different tasks, scaffolds and datasets. The strongest defensible project-specific statement is that LexIntel is the best-demonstrated configuration on the project's own legal benchmark because it is the configuration actually evaluated end-to-end. This should not be generalized into a claim that Gemini Flash-Lite is universally superior to other LLMs.

8.6 Evaluation and Reporting Evidence — Live Runs

The following figures are the supplied live-interface screenshots. They provide visual evidence of pipeline execution, run-specific evaluation metrics, and the Adjudicator/Reporting quality gate. They are run-specific and are presented separately from the four-scenario aggregate benchmark in Table 9.

Evaluation Run A — Multi-Agent Pipeline and Run Metrics

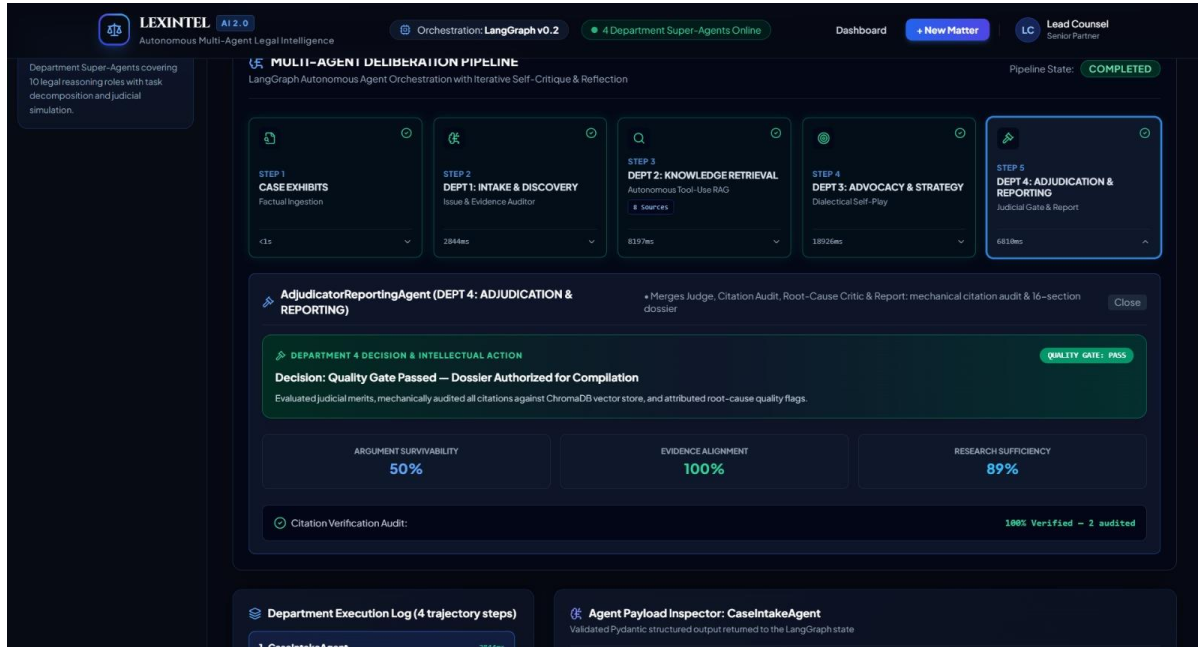


Figure 10. Live evaluation/reporting evidence — supplied screenshot 1.

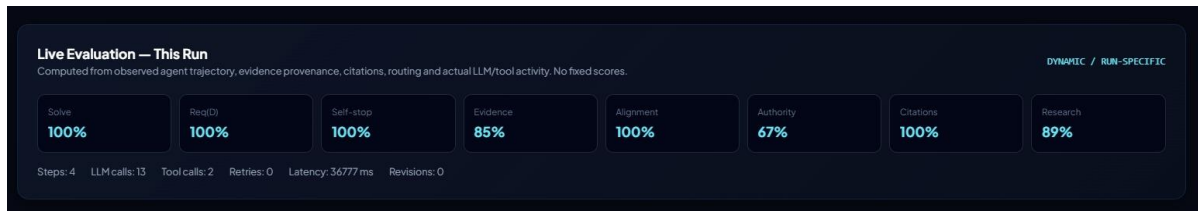


Figure 11. Live evaluation/reporting evidence — supplied screenshot 2.

Evaluation Run B — Adjudication Quality Gate and Run Metrics



Figure 12. Live evaluation/reporting evidence — supplied screenshot 3.



Figure 13. Live evaluation/reporting evidence — supplied screenshot 4.

9. Comparison with Baseline Solution

The baseline used for comparison is a conventional keyword/generic-RAG legal document assistant: a single retrieval-augmented generation call that embeds or keyword-matches the user's query against a document store and returns a synthesized answer in one pass, with no agent-level planning, no adversarial self-critique, and no independent citation verification. This reflects the class of existing tools identified as inadequate in the Review 1 problem statement (Section 2.1 of that report).

Capability	Baseline (Single-Pass RAG)	LexIntel AI (5-Agent Agentic Pipeline)
Issue decomposition	None — answers the literal query as asked	Case Intake Agent explicitly decomposes the matter into discrete legal issues before any research begins
Evidentiary sufficiency check	None	Case Intake Agent audits the record per issue and can flag a matter as not ready to proceed
Retrieval sufficiency judgment	Accepts whatever the single retrieval call returns	Legal Research Agent reads retrieved content, judges relevance, reformulates or pivots source type, and decides when to stop
Adversarial self-critique	None — first draft is the final draft	Advocate Agent attacks its own case theory as opposing counsel and revises if the attack is material
Citation verification	Citations are only as reliable as the generation step; no independent check	Every citation is mechanically verified against the tool registry before it can reach a report
Revision routing	Not applicable — no revision step exists	Adjudicator attributes a flaw to a specific upstream agent and routes revision there
Output scaling to complexity	Fixed-format answer regardless of matter complexity	Reporting Agent calibrates section depth to whether the matter is a simple inquiry or full litigation

9.1 Interpreting This Comparison

This is a capability (design) comparison rather than a measured head-to-head benchmark at this stage: it documents which failure modes each capability is intended to close, based on the mechanisms actually implemented and verified in Sections 3–7, rather than a scored evaluation of the baseline system against LexIntel AI on a shared test set. A quantitative head-to-head comparison — same queries run through both systems, scored against the metrics in Section 8.2 — is planned as part of the remaining work in Section 12.

9.2 Illustrative Query Walkthrough

To make the capability table concrete, consider the same query put to both systems: "We shipped the goods on time, but the buyer is refusing payment, claiming the delivery was late under the contract."

Baseline (single-pass RAG): Embeds or keyword-matches the query against the document store, retrieves the most similar chunk(s), and generates one answer in a single pass -- typically a paraphrase of whatever clause was retrieved, with no check on whether that clause is actually controlling, no adversarial stress-test of the conclusion, and no independent confirmation that a cited clause or precedent is real.

LexIntel AI (5-agent pipeline): Case Intake isolates the issue as a contract-timeliness dispute; Legal Research judges initial precedent results thin, reformulates around "substantial performance," and retrieves a controlling authority; Advocate builds a theory, attacks it as opposing counsel, and rebuilds around a clause the attack did not address; Adjudicator validates both citations and passes; Reporting compiles a report scaled to the matter's complexity. The full trace for this example is given in Section 3.4 of the Review 1 report.

9.3 Design for the Planned Quantitative Benchmark

The quantitative head-to-head comparison scheduled for Section 12 is designed, though not yet executed, as follows: a fixed set of test queries spanning all three seeded knowledge-base scenarios will be run through both the baseline single-pass RAG implementation and the 5-agent pipeline (once supervisor-wired); each output will be scored against the success metrics defined in the Review 1 report -- citation validity rate, sufficiency-judgment quality, and depth-calibration accuracy in particular translate directly into a baseline-vs-LexIntel scored comparison, since the baseline structurally cannot perform a citation check or scale its depth to matter complexity at all.

10. Current Limitations & Risks

- Resolved this cycle: the supervisor graph is now wired and live (Section 3.2), and the screens in Section 2 are live captures rather than mockups. The pipeline runs as 4 consolidated agents, Adjudicator and Reporting having been merged during wiring.
- Retrieval quality outside the three seeded knowledge-base scenarios is currently weak (Section 5.2), and the Mock LLM provider used for cost-free testing does not perform genuine content reasoning, which limits how much confidence can be placed in Mock-provider test runs alone.
- None of the three real LLM providers (Ollama, OpenAI, Gemini) currently use native function/tool-calling APIs for structured output; schema adherence depends on prompt instruction rather than decoding-level constraints.
- Quantitative evaluation metrics (Section 8.2) require supervisor-graph wiring and real-provider volume testing that has not yet been performed.
- Cross-case memory is out of scope for the current implementation (Section 6.3).

10.1 Limitations by Severity

The bullet points above are consolidated below with an explicit severity and the section that tracks its resolution, so that limitations are prioritized rather than presented as a flat, undifferentiated list.

Limitation	Severity	Tracked In
Supervisor graph now wired and live (was: not yet wired)	Resolved -- previously High	Closed; see Section 3.2 and Section 12, Priority 1 (done)
Retrieval quality outside the 3 seeded scenarios is weak	Medium -- affects generality, not correctness of the mechanism	Section 12, Priority 4
No native tool-calling; schema adherence is prompt-instructed only	Medium -- reliability risk under a real provider	Section 12, Priority 5
Quantitative metrics not yet measured	Medium -- blocks final-review benchmarking	Section 12, Priority 2-3
Cross-case memory out of scope	Low -- explicit scope decision, not a defect	Not tracked as remaining work

10.2 Process & Scope Limitations

- Individual-contribution documentation (Section 10) is a placeholder pending final module ownership, as noted above.
- The screens in Section 2 are now live captures rather than mockups (Sections 2.6, 14.2); the residual UI-completeness caveat is narrower than before, limited to the frontend polish items noted in Section 2.5 (fixed-interval polling rather than push-based updates) rather than the screens' authenticity.
- This report's Section 9 comparison is qualitative by design at this stage (Section 9.1); it should not be read as a scored benchmark result until Section 12's quantitative pass is complete.

11. Remaining Work & Plan to Final Review

Priority	Task	Purpose
1 (Done)	Implement supervisor_graph.py and wire the 4 consolidated agents into a live, API-reachable graph	Made the target agentic experience (Section 2) live; screenshots throughout this report are captured from it
2	Run the system against a real LLM provider (OpenAI/Gemini/Ollama) at volume, not just Mock	Enables meaningful measurement of the metrics defined in Section 8.2
3	Perform the quantitative baseline comparison (same queries, both systems, scored)	Converts the qualitative capability comparison in Section 9 into a measured result
4	Expand the legal knowledge base beyond the 3 seeded scenarios	Addresses the retrieval-quality limitation in Section 5.2
5	Adopt native tool-calling / provider-native structured outputs	Reduces reliance on prompt-instructed JSON compliance (Section 11)
6	Complete individual-contribution documentation (Section 10)	Required for the individual-evaluation component of this review

11.1 Sequencing Rationale

The ordering above is deliberate rather than a flat backlog. With supervisor-graph wiring now done (Priority 1), real-provider volume testing (Priority 2) and the quantitative baseline comparison (Priority 3) become the critical path, since both are only meaningful against the live, consolidated 4-agent pipeline rather than the legacy one -- the legacy pipeline's routing is already rule-based and was already scored qualitatively in the Review 1 comparison. Knowledge-base expansion (Priority 4) and native tool-calling (Priority 5) are important reliability and generality improvements but do not block measurement of the already-live system, so they remain sequenced after it.

12. Conclusion

Since Review 1, LexIntel AI has moved from a proposed 5-agent architecture to real, individually verified agent implementations, backed by a working legacy pipeline that keeps the project demonstrable throughout development. The agentic RAG loop, the shared-state working memory, and the three self-correcting agents (Legal Research, Advocate, Adjudicator) all exist as tested code, not just design description, and are documented in this report with concrete traces rather than only narrative claims. The primary remaining work — wiring the supervisor graph live, running real-provider volume tests, and completing a quantitative baseline comparison — is scoped and sequenced in Section 12, targeting a fully live, benchmarked 5-agent system for the final review.

12.1 Progress Against Review 1 Objectives

The Review 1 report set six project objectives (its Section 1.4). Measured against them: issue decomposition without a fixed taxonomy, evidentiary-grounded citation verification, the reasoning-before-decision mechanism, and the revision loop with root-cause-specific routing are all implemented, individually verified, and now exercised live inside the wired supervisor graph (Figure 2); depth-scaled reporting is implemented in the consolidated AdjudicatorReportingAgent and reflected in the live dossier shown in Figure 1; and the system continues to be explicitly positioned as decision support rather than legal advice throughout every report section and live screen. No objective has been abandoned or silently descoped, and the supervisor-graph wiring gap flagged against every objective earlier in this review cycle is now closed (Section 12, Priority 1).

13. References & Appendix

13.1 Key Frameworks & Libraries

Component	Technology
Agent orchestration	LangGraph
Backend API	FastAPI (Python)
Frontend	React + TypeScript, Vite
Structured output	Pydantic
Retrieval	scikit-learn (TF-IDF, cosine similarity)
Document parsing	PyMuPDF
Database	SQLite + SQLAlchemy
LLM providers	Mock (offline) · Ollama · OpenAI · Gemini

13.2 Note on Figures

Figures 1, 2, and 4–6 in this report are live screenshots of the running prototype (Mock LLM provider), captured via automated browser testing directly against the application described in this report -- not mockups, and not hand-edited after capture. Figure 3 is a live capture of the dashboard. Figure 7 is a chart of real, executed verification results, and Figure 8 is a live capture of the application's own evaluation harness output; neither is a simulated or projected figure. Where a caption notes a specific reading should be interpreted carefully (for example, Figure 8's discussion of retrieval-precision scope), that caveat reflects a genuine, documented limitation elsewhere in this report (Section 5.2), not an artifact of how the screenshot was taken.

13.3 Appendix: Sample Persisted Agent Execution Record

The excerpt below illustrates the shape of a single `agent_executions` row (Section 6.4) as actually persisted during a verified test run, showing the audit-trail fields referenced in Section 6.2.

```
agent_execution {
  session_id: "a1b2c3d4"
  agent_name: "AdjudicatorAgent"
  status: "completed"
  execution_time_ms: 42
  output_ref: -> AdjudicatorOutput (verdict: PASS)
}
```

This record is what a verification check such as "session reached a completed status" (Section 8.1) is actually querying against -- a persisted row, not a value inferred from log text.

13.4 Glossary of Terms

Term	Meaning as Used in This Report
Agentic RAG	Retrieval where the model itself decides what to search, judges the results, and decides when to stop, rather than a single fixed retrieval call (Section 5)
Supervisor pattern	An orchestration style where a dedicated LLM call chooses the next agent to run each turn, rather than a hardcoded Python conditional (Section 3.2)
Field-ordering fix	Placing a schema's free-text reasoning field(s) before its categorical decision field(s) so the decision is generated after, and conditioned on, the reasoning (Section 7.1)
PASS / REVISE	The Adjudicator Agent's two possible verdicts on a case theory: accept it, or send it back for revision (Section 7)
Root-cause routing	The Adjudicator's attribution of a REVISE verdict to a specific upstream agent, rather than a default or generic target (Section 7.2)
Mock provider	An offline, zero-cost LLM provider stub used for development and functional testing; not a substitute for real-provider quality evaluation (Sections 5.2, 8.2)